

JXPAMG 1.0 用户指南

JXPAMG 开发团队

2021 年 7 月 24 日

目 录

1	简介	1
1.1	软件架构	2
1.2	下载与安装	2
1.3	使用入门	3
1.3.1	AMG 作为单独解法器	3
1.3.2	AMG 作为预条件子	3
1.3.3	α Setup-AMG 解法器	6
1.4	示例程序	6
1.5	联系方式	7
2	矩阵向量数据结构	8
2.1	预备知识: 稀疏矩阵 CSR 存储格式	8
2.2	矩阵结构体	8
2.2.1	串行矩阵	8
2.2.2	并行矩阵	9
2.3	JXPAMG 中的向量结构体	13
2.3.1	串行向量	13
2.3.2	并行向量	13
3	算法介绍	15
3.1	Krylov 子空间迭代方法	15
3.2	C-AMG: 经典 AMG 算法类	15
3.3	α Setup-AMG: 基于自适应 Setup 的 AMG 算法	18
3.4	AMG- α PCTL: 基于物理量粗化的自适应两层算法	18
3.5	AMG-ILU- α Combined: 基于 AMG 和 ILU 的自适应组合型算法	19
3.6	AMG- α BSchur: 基于 AMG 的块 Schur 补算法	20
3.7	退化为单层情形	20

§1 简介

JXPAMG (**P**arallel **AMG** Solver Developed by **JiuSuo** (IAPCM) and **XTU**) 是由北京应用物理与计算数学研究所和湘潭大学联合研发的一个应用特征感知的并行代数多重网格 (AMG) 解法器软件包, 它主要用于求解实际复杂数值模拟应用中出现的大规模稀疏线性代数方程组.

JXPAMG 的主要特色是以经典 AMG 算法为基础 [1, 2], 集成了若干针对复杂应用特征而发展的特征型 AMG 算法及并行实现策略. 包括: 基于物理量粗化的两层算法 (PCTL) [3] 及其自适应版本 (α PCTL) [4]、基于 ILU 和 AMG 的对称与非对称的组合型预条件算法 [5]、基于代数界面粗化的 AI-AMG 法 [6]、基于自适应 Setup 的并行 α Setup-AMG 法 [7, 8] 以及基于 Schur 补的多种块型预条件算法 [9]. 同时, JXPAMG 实现了常用的 Krylov 子空间迭代方法 [11], 将 AMG 算法视为其预条件子使用. 在经典 AMG 算法的实现中, JXPAMG 继承了劳伦斯利弗莫尔国家实验室 (LLNL) 开发的并行 AMG 解法器 BoomerAMG [10] 的部分代码.

JXPAMG 可以适配国产高性能计算机系统及国产自主 CPU 系统, 已经在多个国家超算中心的超级计算机上进行了部署和测试, 具有良好的并行可扩展性能. 通过并行编程框架 JASMIN [12] 和 JAUMIN [13] 的并行数值代数库 JPSOL 解法器, JXPAMG 已经成功应用于武器物理、激光聚变、工程力学、电磁环境、核能开发、燃烧模拟等领域的十多个应用软件, 成为这些领域很多应用的默认线性解法器, 支撑这些实际应用在国产超级计算机的数万至数十万 CPU 核上完成数亿到百亿自由度的高效隐式计算.

§1.1 软件架构

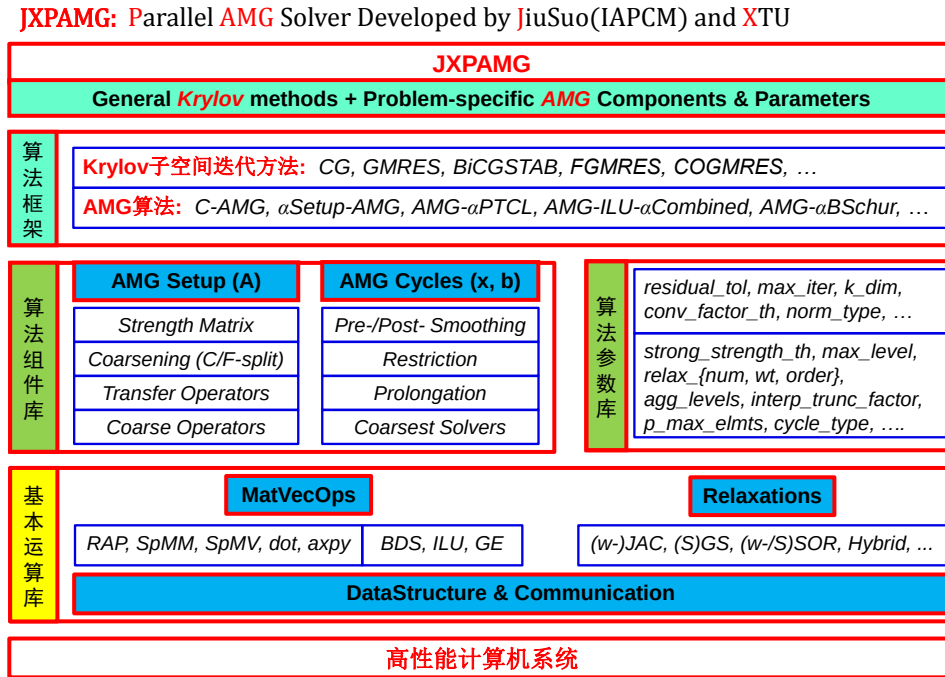


图 1 JXPAMG软件架构

JXPAMG 采用面向复用的软件设计思想. 软件架构包含算法层、组件层和基本运算层, 如图 1 所示. 其中算法层包括通用的 Krylov 子空间迭代方法和各类 AMG 算法, 以及它们的组合. 组件层包含了 AMG 算法的基本组件及相关参数, 包括 Setup 阶段的连通强度矩阵、网格粗化、层间转换算子、粗网格矩阵的构造, 迭代求解阶段 (Cycle) 的光滑过程、插值和限制过程、粗网格求解等. 基本运算层封装算法层和组件层所需的矩阵与向量操作, 以及这些基本操作并行实现所需的数据结构和数据通信等. 基于层次化架构, JXPAMG 实现了算法与实现的分离.

§1.2 下载与安装

用户可访问网址 <http://www.multigrid.org/solver/jxpamg.html> 下载 JXPAMG. 编译与安装的步骤如下:

1. 修改文件 makefile.pub 中的 JXPAMGDIR 和 MPIDIR, 分别改为 JXPAMG 的当前路径和 MPI 的安装路径;
2. 在 JXPAMG 目录下, 利用 make 或 make redo 生成 lib/ 下的静态库 libJXPAMG.a;
3. 修改 makefile.pub 中的安装路径 DEST (默认值 /usr/local/jxpamg-1.1), 利用 make install 安装; 或者利用 make DEST=[install-dir] install 直接安装.

4. 若需线程配置, 需打开 `makefile.pub` 中的 `JXPAMG_USE_OMP`, 值为 YES; 若 `int` 或 `double` 不能满足要求, 需配置 `makefile.pub` 中的 `JX_USE_BIG_INT` 或 `JX_USE_BIG_DOUBLE`, 取值为 YES.

注 1.1 由于 *MPI* 版本的不同导致 *MPI* 静态库的名称不同, 此时需要修正 *makefile.pub* 中的 *MPILIB*.

§1.3 使用入门

§1.3.1 AMG 作为单独解法器

```
// 创建解法器结构体
JX_PAMGCreate(&amg_solver);
// 执行启动阶段
if (max_levels != 1)
    JX_PAMGSetup(amg_solver, (JX_ParCSRMatrix)par_matrix);
// 执行求解阶段
JX_PAMGSolve(amg_solver, (JX_ParCSRMatrix)par_matrix,
              (JX_ParVector)par_rhs, (JX_ParVector)par_sol);
// 释放内存
JX_PAMGDestroy(amg_solver);
```

当系数矩阵是对称的 M 矩阵时, AMG 解法器通常是非常有效的, 其基本组件以及相关参数的设置, 参见 §3.2.

§1.3.2 AMG 作为预条件子

在实际应用中, AMG 通常是作为 Krylov 子空间迭代方法的预条件子使用. JXPAMG 实现了若干 Krylov 子空间类方法, 详见 §3.1.

下面, 以预条件共轭梯度方法 (PCG) 为例, 给出相关预条件子的用法. 其它 Krylov 子空间方法是类似的, 可以参见示例程序 (见 §1.4).

- 经典 AMG 预条件.

```
// 创建预条件子结构体
JX_PAMGCreate(&amg_solver);
// 创建解法器结构体
JX_ParCSRPCGCreate(comm, &solver);
// 设置预条件子
JX_PCGSetPrecond(solver, (JX_PtrToSolverFcn)JX_PAMGPrecond,
                  (JX_PtrToSolverFcn)JX_PAMGSetup, amg_solver);
// 执行预条件子的启动阶段
```

```

JX_PAMGSetup(amg_solver, (JX_ParCSRMatrix)par_matrix);
// 执行解法器的启动阶段
JX_PCGSetup(solver, (JX_Matrix)par_matrix,
            (JX_Vector)par_rhs, (JX_Vector)par_sol);
// 执行解法器的求解阶段
JX_PCGSolve(solver, (JX_Matrix)par_matrix,
            (JX_Matrix)par_matrix, (JX_Vector)par_rhs, (JX_Vector)par_sol);
// 释放内存
JX_PAMGDestroy(amg_solver);
JX_ParCSRPCGDestroy(solver);

```

- AMG- α PCTL 预条件.

```

// 创建预条件子结构体
JX_3tAPCTLDataInitialize(&precond);
// 创建解法器结构体
JX_ParCSRPCGCreate(comm, &solver);
// 设置预条件子
JX_PCGSetPrecond(solver, (JX_PtrToSolverFcn)JX_3tAPCTLmgPrecond,
                (JX_PtrToSolverFcn)NULL, precond);
// 执行预条件子的启动阶段
JX_3tAPCTLSetup4mgJasmin(precond, (JX_ParCSRMatrix)par_matrix);
// 执行解法器的启动阶段
JX_PCGSetup(solver, (JX_Matrix)par_matrix,
            (JX_Vector)par_rhs, (JX_Vector)par_sol);
// 执行解法器的求解阶段
JX_PCGSolve(solver, (JX_Matrix)par_matrix,
            (JX_Matrix)par_matrix, (JX_Vector)par_rhs, (JX_Vector)par_sol);
// 释放内存
JX_3tAPCTLDestroy4mgJasmin(precond);
JX_ParCSRPCGDestroy(solver);

```

- AMG-ILU- α Combined 预条件.

```

// 创建预条件子结构体
JX_CombinedPrecondDataCreate(&combined_solver, comm);
// 创建解法器结构体
JX_ParCSRPCGCreate(comm, &solver);

```

```

// 设置预条件子
JX_PCGSetPrecond(solver, (JX_PtrToSolverFcn)JX_CombinedPrecondDataSolve,
                  (JX_PtrToSolverFcn)JX_CombinedPrecondDataSetup, combined_solver);
// 执行预条件子的启动阶段
JX_CombinedPrecondDataSetup(combined_solver, (JX_ParCSRMatrix)par_matrix);
// 执行解法器的启动阶段
JX_PCGSetup(solver, (JX_Matrix)par_matrix,
             (JX_Vector)par_rhs, (JX_Vector)par_sol);
// 执行解法器的求解阶段
JX_PCGSolve(solver, (JX_Matrix)par_matrix,
             (JX_Matrix)par_matrix, (JX_Vector)par_rhs, (JX_Vector)par_sol);
// 释放内存
JX_CombinedPrecondDataDestroy(combined_solver);
JX_ParCSRPCGDestroy(solver);

```

- AMG- α BSchur 预条件.

```

// 创建预条件子结构体
JX_3tAPCTLDataInitialize(&schur_solver);
// 创建解法器结构体
JX_ParCSRPCGCreate(comm, &solver);
// 设置预条件子
JX_PCGSetPrecond(solver, (JX_PtrToSolverFcn)JX_3tABSC1mgPrecond,
                  (JX_PtrToSolverFcn)NULL, schur_solver);
// 执行预条件子的启动阶段
JX_3tABSC1Setup4mgJasmin(schur_solver, (JX_ParCSRMatrix)par_matrix);
// 执行解法器的启动阶段
JX_PCGSetup(solver, (JX_Matrix)par_matrix,
             (JX_Vector)par_rhs, (JX_Vector)par_sol);
// 执行解法器的求解阶段
JX_PCGSolve(solver, (JX_Matrix)par_matrix,
             (JX_Matrix)par_matrix, (JX_Vector)par_rhs, (JX_Vector)par_sol);
// 释放内存
JX_3tAPCTLDestroy4mgJasmin(schur_solver);
JX_ParCSRPCGDestroy(solver);

```

- 无预条件情形.

```

// 创建解法器结构体

```

```

JX_ParCSRPCGCreate(comm, &solver);
// 执行解法器的启动阶段
JX_PCGSetup(solver, (JX_Matrix)par_matrix,
            (JX_Vector)par_rhs, (JX_Vector)par_sol);
// 执行解法器的求解阶段
JX_PCGSolve(solver, (JX_Matrix)par_matrix,
            (JX_Matrix)par_matrix, (JX_Vector)par_rhs, (JX_Vector)par_sol);
// 释放内存
JX_ParCSRPCGDestroy(solver);

```

§1.3.3 α Setup-AMG 解法器

```

// 创建解法器结构体
JX_ParAdpSetupAMGCreate(comm, &solver);
for (pro_id = 0; pro_id < pro_num; pro_id++) {
    // 执行启动阶段
    JX_ParAdpSetupAMGSetup(solver,
        (JX_Matrix)par_matrix, (JX_Vector)par_rhs, (JX_Vector)par_sol);
    // 执行求解阶段
    JX_ParAdpSetupAMGSolve(solver, (JX_Matrix)par_matrix,
        (JX_Matrix)par_matrix, (JX_Vector)par_rhs, (JX_Vector)par_sol);
}
// 释放内存
JX_ParAdpSetupAMGDestroy(solver);

```

当求解的是线性代数方程组序列 (如对于求解时间相关的非线性偏微分方程组问题等) 时, 推荐使用 α Setup-AMG 算法, 其相关参数设置, 参见 §3.3.

§1.4 示例程序

上述使用方法可以参见 JXPAMG 的示例程序 [example/solver_strong.c](#) (包括经典 AMG 解法器以及经典 AMG, AMG-ILU- α Combined 和无预条件的 Krylov 子空间迭代方法), [example/apctl_mg.c](#) (AMG- α PCTL 预条件的 Krylov 子空间迭代方法), [example/schur_mg.c](#) (两类 AMG- α BSchur 预条件的 Krylov 子空间迭代方法) 和 [example/solver_asetup.c](#) (α Setup-AMG 解法器). 这些示例程序仅需用户提供系数矩阵文件和右端向量文件, 文件格式分别为

1. 系数矩阵文件: CSR 格式 (详见第 §2.1 节), 指标起始为 1 而不是 0,
 - 第 1 行为矩阵规模 n ,
 - 第 2 至第 $n+2$ 行为矩阵的行管理数组 $IA(i)$, $i=1, \dots, n+1$,

- 第 $n+3$ 至第 $n+2+nz$ 为矩阵非零元的列号数组 $JA(i)$, $i=1,\dots,nz$,
- 第 $n+3+nz$ 至第 $n+2+2*nz$ 为矩阵非零元的数值数组 $AA(i)$, $i=1,\dots,nz$,

其中 $nz=IA(n+1)-1$.

2. 右端向量文件:

- 第 1 行为向量规模 n ,
- 第 2 至第 $n+1$ 行为向量的各分量值.

§1.5 联系方式

技术支持及错误反馈, 请联系 yuexq@xtu.edu.cn (岳孝强) 和 xwxu@iapcm.ac.cn (徐小文).

§2 矩阵向量数据结构

§2.1 预备知识: 稀疏矩阵 CSR 存储格式

CSR (Compressed Sparse Row) 格式是存储稀疏矩阵的最常用格式, 该格式需用到以下三个一维数组:

1. $AA(i)$, $i = 1(1)nz$: 实型数组, 按行存放矩阵 A 中的所有非零元素.
2. $JA(i)$, $i = 1(1)nz$: 整型数组, 依次存放数组 AA 中对应元素的列号.
3. $IA(i)$, $i = 1(1)n + 1$: 整型数组, 依次存放各行在数组 AA 或 JA 中存储的首地址. 当 j 从 $IA(i)$ 到 $IA(i + 1) - 1$ 变化时, $AA(j)$ 即可遍历矩阵 A 第 i 行的所有非零元素.

注 2.1 数组 AA 中按行存放矩阵 A 的非零元素时, 每一行非零元素的顺序可以是任意的, 通常按照按列号从小到大的顺序来存放, 有时为方便访问对角线元素, 在每一行也常按照先对角线元素后非对角线元素的顺序来存放.

注 2.2 若矩阵 A 的第 i 行没有非零元素, 则有 $IA(i) = IA(i + 1)$.

例 2.1 矩阵

$$A = \begin{pmatrix} 1. & 0. & 2. & 0. \\ 0. & 3. & 0. & 4. \\ 5. & 0. & 6. & 7. \\ 8. & 0. & 0. & 9. \end{pmatrix}$$

的 CSR 存储格式为

表 1 矩阵 A 的 CSR 存储格式

AA	1.	2.	4.	3.	6.	5.	7.	8.	9.
JA	1	3	4	2	3	1	4	1	4
IA	1	3	5	8	10				

§2.2 矩阵结构体

§2.2.1 串行矩阵

在介绍并行矩阵数据结构之前, 先引入串行矩阵结构体 `jx_CSRMatrix`.

```
typedef struct
{
    double *data; // 指向非零元数组存储空间首地址的指针
    int *i; // 指向行索引数组存储空间首地址的指针
```

```

int      *j;      // 指向列指标数组存储空间首地址的指针
int      num_rows; // 行数
int      num_cols; // 列数
int      num_nonzeros; // 非零元个数
int      *rownnnz; // 指向非零行(行非零元个数不为 0)存储空间首地址的指针
int      num_rownnnz; // 非零行的数量
int      owns_data;
// 标记值. 1: 表示该串行矩阵有自己的 data 数组和 j 数组, 0: 表示没有
} jx_CSRMatrix;

```

jx_CSRMatrix 采用压缩稀疏行 (CSR) 数据结构, 它包括整数 num_rows (矩阵的行数)、整型数组 i、j 和大小为矩阵非零元个数的实型数组 data (含矩阵的所有非零元数值) 组成. 同一行的元素排在一起, 尽管它们不需行内排序. 但是, 各行是需要按序排列的, 即第 $k+1$ 行必须紧跟第 k 行. j 是一个与 data 相同长度的整数数组, 它的第 l 个元素为 data 的第 l 个元素的列索引. i 是一个长度为 num_rows+1 的整型数组, 其第 k 个元素指向数组 data 和 j 中第 k 行的第一个非零元的位置.

§2.2.2 并行矩阵

JXPAMG 并行矩阵数据结构采用基于数据剖分的分布式结构体. 以 N 阶矩阵 A 为例, 它以连续的“行块”按如下方式将矩阵划分到 p 个进程:

$$\begin{pmatrix} A_1 \\ A_2 \\ \vdots \\ A_p \end{pmatrix}$$

其中子块 A_k 分布在进程 k 上, 它包含第 $r_k, r_k+1, \dots, r_{k+1}-1$ 行, 这里 $[r_1, r_2, \dots, r_p, r_{p+1}]$ 为 A 的全局分划数组, $r_1 = 1, r_{p+1} = N + 1$.

基于上述分布式数据剖分, JXPAMG 设计了并行稀疏矩阵结构 jx_ParCSRMatrix. 一个 jx_ParCSRMatrix 由 p 个子块 A_k ($k = 1, \dots, p$) 构成, 其中 A_k 局部存储在进程 k 上. 每个 A_k 分为两个矩阵 D_k (diag) 和 O_k (offd). D_k 是 n_k 阶方阵, 其中 $n_k = r_{k+1} - r_k$ 是属于进程 k 的行数, D_k 包含所有系数 a_{ij} , 其中 $r_k \leq i, j \leq r_{k+1} - 1$, 即属于局部行索引集的列索引. 第二个矩阵 O_k 包含 A_k 的列索引 j 属于其它进程上 ($j < r_k$ 或 $j \geq r_{k+1}$) 的元素. 这两个矩阵均以 jx_CSRMatrix 存储. D_k 是通常意义下的 CSR 矩阵, 而 O_k 则通常极其稀疏 (有很多零列和零行), 为了提高效率, 所有非零列索引均重新编号. 因此, 需要生成一个长度为 n_{O_k} 的整型数组, 它定义了局部到全局的列索引映射, 其中 n_{O_k} 是 O_k 的非零列数. 该数组记为 col_map_Ok (col_map_offd).

jx_ParCSRMatrix 结构体为:

```
typedef struct
```

```

{
    MPI_Comm          comm;           //通信域
    int                global_num_rows; //整个矩阵的行数
    int                global_num_cols; //整个矩阵的列数
    int                first_row_index; //本地矩阵中第一行对应于整体矩阵的行号
    int                first_col_diag;  //本地矩阵对角块第一列对应于整体矩阵的列号
    int                last_row_index;  //本地矩阵中最后一行对应于整体矩阵的行号
    int                last_col_diag;   //本地矩阵对角块最后一列对应于整体矩阵的列号
    jx_CSRMatrix       *diag;           //指向本地矩阵对角块矩阵存储空间的首地址指针
    jx_CSRMatrix       *offd;           //指向本地矩阵非对角块存储空间的首地址指针
    int                *col_map_offd;    //指向非对角块各非零元列编号到整体矩阵列编号的
                                         //映射数组的存储空间首地址的指针

    int                *row_starts;      //指向行剖分数组存储空间首地址的指针
    int                *col_starts;      //指向列剖分数组存储空间首地址的指针
    jx_ParCSRCommPkg   *comm_pkg;        //指向矩阵 A 的通信包的首地址的指针
    jx_ParCSRCommPkg   *comm_pkgT;       //指向矩阵 AT 的通信包的首地址的指针
    int                owns_data;
    //标记值, 1: 表示该并行矩阵有自己的对角块和非对角块, 0: 表示没有
    int                owns_row_starts;
    //标记值, 1: 表示该并行矩阵有自己的行分划, 0: 表示没有
    int                owns_col_starts;
    //标记值, 1: 表示该并行矩阵有自己的列分划, 0: 表示没有
    // 以上三个标记值将影响最终并行矩阵的释放, 因此这里需要特别注意,
    // 若不存在自己对应的成员, 如: 行剖分数组共用别的并行向量或矩阵的指针,
    // 且两者标记值均为 1, 则会出现重复释放的问题.
    int                num_nonzeros;     // 整个矩阵的非零元个数
    // 以下几个为不常用的成员变量 (在指定分划数组时使用)
    double             d_num_nonzeros;
    // 整个矩阵的非零元个数, 但注意这里的变量类型为实型.
    int                *rowindices;
    double             *rowvalues;
    int                getrowactive;
    jx_IJAssumedPart   *assumed_partition; /* only populated if no_global_partition
                                         option is used (compile-time option) */
} jx_ParCSRMatrix;

```

注 2.3 *col_map_offd* 表示非对角块各非零列 (有非零元的列) 的整体编号, 此时

1. 数组长度: 非对角块非零列的个数;

2. *offd_j*: 存放的是非对角块各非零元对应的局部列号. (同样也可理解为非对角块各非零元对应的整体列号在 *col_map_offd* 数组中的位置.) 注意这时, 非对角块指的是并行矩阵中除去对角块及零列后得到的矩阵.

注 2.4 *jx_ParCSRMatrix* 结构体与 *HYPRE* 的并行稀疏矩阵格式 *hypr_ParCSRMatrix* 和 *PETSc* 的稀疏矩阵格式 *Mat_MPIAIJ* 类似.

图 2 以 11 阶矩阵来说明上述数据结构. 矩阵分布在 3 个进程上, 进程 1 和 2 上各有 4 行, 进程 3 只有 3 行. 4 阶矩阵 D_1 和 D_2 以及 3 阶矩阵 D_3 用方框表示, 其余元素被压缩到 4×3 矩阵 O_1 ($\text{col_map_O1}=(5,6,8)$), 4×4 矩阵 O_2 ($\text{col_map_O2}=(1,2,4,9)$), 3×4 矩阵 O_3 ($\text{col_map_O3}=(3,4,5,8)$). 由于 O_p 通常非常稀疏, 还可通过引入一个行映射来进一步提高效率, 该行映射将压缩零行而对非零行重新编号.

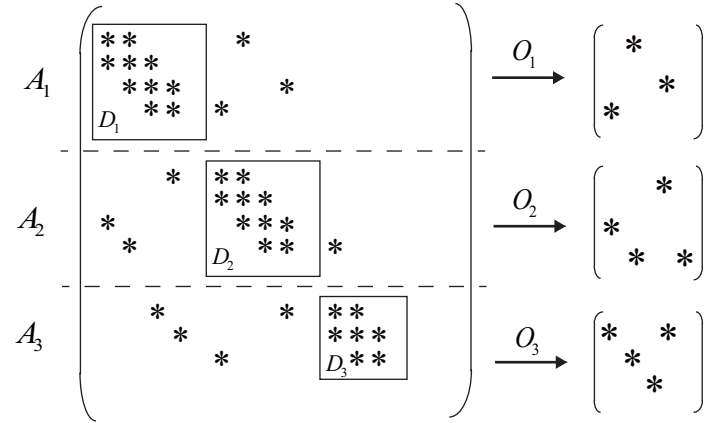


图 2 分布在 3 个进程上的 *jx_ParCSRMatrix* 示例, 各进程的局部矩阵 D_1 、 D_2 和 D_3 用方框表示, 其它元素则压缩到子块 O_1 、 O_2 和 O_3 .

例 2.2 矩阵

$$A = \begin{pmatrix} 4 & -1 & -1 & 0 \\ -1 & 4 & 0 & -1 \\ -1 & 0 & 4 & -1 \\ 0 & -1 & -1 & 4 \end{pmatrix}.$$

当进程数为 2 时, 每个进程对应的并行矩阵 *jx_ParCSRMatrix* 中几个常用成员的具体取值.

表 2 两个进程对应并行矩阵的成员

结构体成员		my_id=0	my_id=1
global_num_rows		4	4
global_num_cols		4	4
first_row_index		0	2
first_col_diag		0	2
last_row_index		1	3
last_col_diag		1	3
diag	data	{4,-1,-1,4}	{4,-1,-1,4}
	i	{0,2,4}	{0,2,4}
	j	{0,1,0,1}	{0,1,0,1}
	num_rows	2	2
	num_cols	2	2
	num_nonzeros	4	4
offd	data	{-1,-1}	{-1,-1}
	i	{0,1,2}	{0,1,2}
	j	{0,1}	{0,1}
	num_rows	2	2
	num_cols	2	2
	num_nonzeros	2	2
col_map_offd		{2,3}	{0,1}
row_starts		{0,2,4}	{0,2,4}
col_starts		{0,2,4}	{0,2,4}
num_nonzeros		12	12

jx_ParCSRMatrix 的通信包是基于矩阵向量乘积的需求而生成的. 考虑矩阵 A 与向量 x 的并行乘积. 进程 k 拥有 A 的第 r_k 行至第 $r_{k+1} - 1$ 行以及 x 的对应分块

$$x_k = (x_{r_k}, \dots, x_{r_{k+1}-1})^\top.$$

为了实现 A 与 x 的乘积, 进程 k 需执行下述运算:

$$A_k x = D_k x_k + O_k \tilde{x}_k,$$

其中 $\tilde{x}_k = (x_{col_map_Ok(1)}, \dots, x_{col_map_Ok(n_{O_k})})^\top$. D_k 和 x_k 的乘积不需任何的通信即可完成, 但是 $\tilde{x}_k$ 的元素归进程 k 的“接收”进程所有. 另一个必要的信息是各进程需要接收的数据量. 通常, 进程 k 拥有其他进程所需的向量元素. 因此, 进程 k 需要知道它所“发送”的进程需要的元素索引.

§2.3 JXPAMG 中的向量结构体

§2.3.1 串行向量

串行向量的结构体 `jx_Vector` 为

```
typedef struct
{
    double    *data;        // 指向元素值数组存储空间首地址的指针
    int        size;        // 向量的长度
    int        owns_data;
    //标记值, 1: 表示该向量有自己的 data, 0: 表示没有
    int        num_vectors;    // 该向量存放的子向量个数
    int        multivec_storage_method; // 多个子向量存放的方法.
    // 存放多个向量时, 这里每个子向量的长度为 size, 且所有元素均存放在 data 数组中
    int        vecstride; // 表示每个子向量中相同编号的元素在 data 中位置之间的偏移量
    int        idxstride; // 表示各个子向量中相邻编号元素在 data 中位置之间的偏移量
} jx_Vector;
```

§2.3.2 并行向量

并行向量的结构体 `jx_ParVector` 为

```
typedef struct
{
    MPI_Comm    comm;        // 通信域
    int          global_size; // 整体向量的长度
    int          first_index; // 当前进程中向量的第一个元素在整体向量中的编号
    int          last_index;  // 当前进程中向量的最后一个元素在整体向量中的编号
    int          *partitioning; // 指向进程分划数组存储空间首地址的指针
    jx_Vector    *local_vector; // 指向当前进程所对应的子向量存储空间首地址的指针
    int          owns_data;
    //标记值, 1: 表示该并行向量有自己的 local_vector, 0: 表示没有
    int          owns_partitioning;
    //标记值, 1: 表示该并行向量有自己的 partitioning, 0: 表示没有
    jx_IJAssumedPart *assumed_partition;
    /* only populated if no_global_partition option
       is used (compile-time option) AND this partition
       needed(for setting off-proc elements, for example) */
} jx_ParVector;
```

例 2.3 向量 $x = (1, 2, 3, 4, 5, 6)^T$. 当进程数为 2 时, 每个进程对应的并行向量 `jx_ParVector` 中几个常用成员的具体取值.

表 3 两个进程对应并行向量的成员

结构体成员		my_id=0	my_id=1
global_size		6	6
first_index		0	3
last_index		2	5
partitioning		{0,3,6}	{0,3,6}
local_vector	data	{1,2,3}	{4,5,6}
	size	3	3
	owns_data	1	1
	num_vectors	1	1
	multivec_storage_method	0	0
	vecstride	3	3
	idxstride	1	1

§3 算法介绍

JXPAMG 实现了若干 Krylov 子空间迭代方法和 AMG 算法.

§3.1 Krylov 子空间迭代方法

JXPAMG 目前实现了共轭梯度 (CG)、重启型广义极小残量 (GMRES(m), 回头数或 Krylov 子空间维数为 m)、flexible GMRES(m)、基于经典 Gram-Schmidt 与重正交的低通信 GMRES(m) (COGMRES(m)) 和双共轭梯度稳定 (BiCGSTAB) 等方法, 这些算法的详细介绍可参考文献 [11], 这里不再赘述.

表 4 Krylov 子空间迭代方法的常用接口

	函数名	功能	默认值
CG	JX_PCGSetMaxIter	设置最大迭代次数	1000
	JX_PCGSetTol	设置迭代收敛相对残差阈值	1.0e-6
	JX_PCGSetTwoNorm	设置收敛范数类型 0: 预条件范数, 1: ℓ_2 范数	0
GMRES	JX_GMRESSetKDim	设置 Krylov 子空间维数	5
	JX_GMRESSetIsCheckRestarted	设置假收敛后是否重算	0
	JX_GMRESSetMaxIter	设置最大迭代次数	1000
	JX_GMRESSetTol	设置迭代收敛相对残差阈值	1.0e-6
BiCGSTAB	JX_BiCGSTABSetMaxIter	设置最大迭代次数	1000
	JX_BiCGSTABSetTol	设置迭代收敛相对残差阈值	1.0e-6

§3.2 C-AMG: 经典 AMG 算法类

该类算法实现了经典 AMG 法的几乎所有算法组件, 包括: 光滑子 (磨光算法)、粗化算法、插值算法、限制算法、粗网格矩阵构造方法和最粗网格层求解方法等.

- 光滑子. 包括典型松弛迭代 w-Jacobi (w-JAC)、Gauss-Seidel (GS)、SGS、w-SOR、w-SSOR 等方法和不完全矩阵分解 ILU 算法. 其中, 松弛型光滑子还包括松弛因子 w 和不同的松弛顺序 (如 CF 序等), GS、SGS、SOR、SSOR 等松弛迭代还包括串行实现和并行混合实现 (块内用串行, 块边界用 Jacobi 型 [14, 15]) 等.
- 粗化算法. 包括经典粗化算法 RS 类, 以及 CLJP [16, 17, 18]、Falgout [10]、PMIS [19]、HMIS [19]、Relaxed-type [20, 21]、代数界面型粗化 [6] 等, 这些粗化算法均可以通过在指定网格层采用 aggressive 策略 [2] 来降低算子复杂度.
- 插值算法. 包括经典插值 [1]、直接插值 [2]、标准插值 [2]、多步插值 [2]、扩展型长距离经典插值 [19] 等. 其中, 直接插值、标准插值、多步插值还包括权重分离情形 (separation weight). 长距离插值通常用于 aggressive 策略、以及 PMIS 和 HMIS 等低复杂度的粗化算法, 此外, 还可以通过截断策略 (truncation) 来降低算子复杂度.

- 粗网格矩阵. 包括经典 Galerkin 型 [1, 2] 和 non-Galerkin 型 [22] 方法.
- 最粗网格层求解方法. 包括直接法 (GE) 和所有光滑算法都可作为最粗网格层解法器.

关于 AMG 算法研究的相关进展, 可参考综述文章 [23, 24].

表 5 给出 C-AMG 结构体 `jx_ParAMGData` 的常用接口, 其中的循环类型见表 6、磨光类型见表 7、粗化类型见表 8 以及插值类型见表 9.

表 5 `jx_ParAMGData` 的常用接口

函数名	功能	默认值
<code>JX_PAMGSetMaxLevels</code>	设置最大网格层数	25
<code>JX_PAMGSetMaxIter</code>	设置最大迭代次数	20
<code>JX_PAMGSetCycleType</code>	设置循环类型	1
<code>JX_PAMGSetTol</code>	设置迭代收敛相对残差阈值	1.0e-7
<code>JX_PAMGSetCoarsenType</code>	设置粗化类型	6
<code>JX_PAMGSetInterpType</code>	设置插值类型	0
<code>JX_PAMGSetPMaxElmts</code>	设置插值算子的截断最大元素个数	0
<code>JX_PAMGSetAggNumLevels</code>	设置 aggressive 粗化层数	0
<code>JX_PAMGSetStrongThreshold</code>	设置强弱连通阈值	0.25
<code>JX_PAMGSetMaxRowSum</code>	设置最大行和参数	0.9
<code>JX_PAMGSetPrintLevel</code>	设置打印输出级别	0
<code>JX_PAMGSetCoarsestSolverID</code>	设置最粗层解法器的类型	9
<code>JX_PAMGSetCycleNumSweeps</code>	设置循环中的磨光次数	1
<code>JX_PAMGSetCycleRelaxType</code>	设置循环中的磨光类型	3

注 3.1 对给定的强弱连通阈值 $\theta_1 \in [0, 1)$ 和最大行和参数 $\theta_2 \in (0, 1]$, 节点 i 的强依赖集 S_i 定义为

- 当 $\theta_2 = 1$, 或 $\theta_2 \in (0, 1)$ 且 $\left| \frac{\sum_{j=1}^n a_{ij}}{a_{ii}} \right| \leq \theta_2$ 时,

$$S_i = \{j \neq i \mid a_{ij} \begin{cases} \geq \theta_1 \max_{k \neq i} \{a_{ik}\}, & a_{ii} < 0 \\ \leq \theta_1 \min_{k \neq i} \{a_{ik}\}, & a_{ii} \geq 0 \end{cases} \}.$$

- 当 $\theta_2 \in (0, 1)$ 且 $\left| \frac{\sum_{j=1}^n a_{ij}}{a_{ii}} \right| > \theta_2$ 时,

$$S_i = \emptyset.$$

表 6 循环类型

cycle_type 的取值	含义
1	V-cycle
2	W-cycle

表 7 磨光类型

relax_type 的取值	含义
0	Jacobi 或 CF-Jacobi
3	混合 Gauss-Seidel
6	混合对称 Gauss-Seidel
9	Gauss 消元法

表 8 粗化类型

coarsen_type 取值	含义
0	CLJP
1	Ruge
11	Ruge 1st pass only
6	Falgout
8	PMIS
10	HMIS
90	RCLJP
91	RRS0
990	CLJP_AI
991	Ruge_AI
96	Falgout_AI
98	PMIS_AI
910	HMIS_AI

表 9 插值类型

interp_type 取值	含义
0	修正的经典插值
3	直接插值 (权重分离)
4	多步插值
5	多步插值 (权重分离)
6	扩展的经典修正插值
8	标准插值
9	标准插值 (权重分离)

§3.3 α Setup-AMG: 基于自适应 Setup 的 AMG 算法

该类算法主要针对求解方程组序列问题而提出, 主要思想是通过设计 Setup 条件, 对序列中不同的方程自适应地采用不同的光滑次数和网格层数, 从而尽可能降低 AMG 算法中 Setup 开销、提升并行可扩展性. 该方法适合时间依赖的非线性偏微分方程应用. α Setup-AMG 的算法组件与 C-AMG 一样.

表 10 给出 α Setup-AMG 结构体 `jx_ParAdpSetupAMGData` 的常用接口.

表 10 `jx_ParAdpSetupAMGData` 的常用接口

函数名	功能	默认值
<code>JX_ParAdpSetupAMGSetIsAdaptive</code>	设置是否自适应	<code>JX_FALSE</code>
<code>JX_ParAdpSetupAMGSetIsAdapSetup</code>	设置是否自适应 Setup	<code>JX_FALSE</code>
<code>JX_ParAdpSetupAMGSetTol</code>	设置迭代收敛相对残差阈值	<code>1.0e-7</code>
<code>JX_ParAdpSetupAMGSetKDim</code>	设置 GMRES 的回头数	<code>5</code>
<code>JX_ParAdpSetupAMGSetTwoNorm</code>	设置 CG 法的收敛范数	<code>0</code>
<code>JX_ParAdpSetupAMGSetSolverID</code>	设置解法器 1: CG, 2: GMRES, 3: BiCGSTAB	<code>2</code>
<code>JX_ParAdpSetupAMGSetPrintLevel</code>	设置打印输出级别	<code>0</code>
<code>JX_ParAdpSetupAMGSetMaxIterSL</code>	设置单层最大迭代次数	<code>1000</code>
<code>JX_ParAdpSetupAMGSetMaxIterReUse</code>	设置重用最大迭代次数	<code>1000</code>
<code>JX_ParAdpSetupAMGGetStatus</code>	输出 α Setup-AMG 的各参数取值	<code>-</code>

注 3.2 关于 `MaxIterSL` 和 `MaxIterReUse` 的设置方法, 参见 [8].

§3.4 AMG- α PCTL: 基于物理量粗化的自适应两层算法

该类算法主要针对辐射流体力学计算中三温能量方程的求解而提出, 主要思想是利用三温耦合的特殊结构, 将电子温度视为粗网格, 将三温耦合方程组化为 4 个标量方程, 采用 AMG 进行求解. 文 [4] 给出一种该算法的自适应策略, 基于块间耦合、块内对角占优的强弱

度量自适应选择不同的策略: 若块内对角占优强, 则该块采用 Jacobi 求解, 否则采用 AMG 求解; 若块间耦合较弱, 则采用对角块预条件子, 否则采用 PCTL 预条件子.

表 11 给出 AMG- α PCTL 结构体 `jx_3tAPCTLData` 的常用接口.

表 11 `jx_3tAPCTLData` 的常用接口

函数名	功能	默认值
<code>JX_3tAPCTLSetNumGroup</code>	设置辐射能群个数	1
<code>JX_3tAPCTLSetA</code>	设置预条件矩阵	-
<code>JX_3tAPCTLSetMaxiter</code>	设置最大迭代次数	100
<code>JX_3tAPCTLSetPrintLevel</code>	设置打印输出级别	0
<code>JX_3tAPCTLSetBlockSmoothType</code>	设置块磨光的类型	BLOCKSMOOTH_BD
<code>JX_3tAPCTLSetThetaWCE</code>	设置电子的弱耦合因子	1.0e-4
<code>JX_3tAPCTLSetThresholdWCE</code>	设置电子的弱耦合阈值	0.5
<code>JX_3tAPCTLSetThetaDDR</code>	设置光子的对角占优因子	0.9
<code>JX_3tAPCTLSetThetaDDE</code>	设置电子的对角占优因子	0.9
<code>JX_3tAPCTLSetThetaDDI</code>	设置离子的对角占优因子	0.9
<code>JX_3tAPCTLSetThresholdDDR</code>	设置光子的对角占优阈值	1.0
<code>JX_3tAPCTLSetThresholdDDE</code>	设置电子的对角占优阈值	1.0
<code>JX_3tAPCTLSetThresholdDDI</code>	设置离子的对角占优阈值	1.0

§3.5 AMG-ILU- α Combined: 基于 AMG 和 ILU 的自适应组合型算法

该类算法主要针对辐射流体力学计算中的多群扩散问题而提出, 主要思想是基于 ILU(k) 和 AMG 在降低多尺度强度和多尺度分布上的互补性, 给出如下自适应策略: 先利用残差阈值判断 ILU(0) 是否合适, 若不合适, 则利用多尺度阈值判断 AMG 是否适用, 否则采用组合型算法求解. 该方法中涉及的 AMG 算法, 均可以采用 C-AMG 或 α Setup-AMG.

表 12 给出 AMG-ILU- α Combined 结构体 `jx_CombinedPrecondData` 的常用接口, 其中的预条件类型见表 13.

表 12 `jx_CombinedPrecondData` 的常用接口

函数名	功能	默认值
<code>JX_CombinedPrecondDataSetPreID</code>	设置预条件的类型	11
<code>JX_CombinedPrecondDataSetDropTol</code>	设置 ILU 的过滤参数	0
<code>JX_CombinedPrecondDataSetInterpType</code>	设置 AMG 的插值类型	0
<code>JX_CombinedPrecondDataSetCoarsenType</code>	设置 AMG 的粗化类型	6
<code>JX_CombinedPrecondDataSetCycleRelaxType</code>	设置 AMG 的磨光类型	3
<code>JX_CombinedPrecondDataSetStrongThreshold</code>	设置 AMG 的强弱连通阈值	0.25
<code>JX_CombinedPrecondDataInitialize</code>	创建 ILU 和 AMG 结构体	-

表 13 预条件的类型

pre_id 的取值	含义
11	AMG-Euclid-AMG
12	Euclid-AMG
13	AMG-Euclid
14	Euclid-AMG-Euclid
15	自适应策略
16	ILU(0)-AMG
17	ILU(0)-AMG-ILU(0)
19	AMG-ILU(0)-AMG

§3.6 AMG- α BSchur: 基于 AMG 的块 Schur 补算法

该类算法主要针对辐射流体力学计算中单、多群扩散方程的求解而提出的两类块 Schur 补算法, 其中子块采用 AMG 算法类求解.

表 14 给出 AMG- α BSchur 结构体 jx_3tAPCTLData 的常用接口.

表 14 jx_3tAPCTLData 的常用接口

函数名	功能	默认值
JX_3tAPCTLSetNumGroup	设置辐射能群个数	1
JX_3tAPCTLSetSubBlocks	设置 AMG- α BSchur 所需的各个子块	-
JX_3tAPCTLSetMaxiter	设置最大迭代次数	100
JX_3tAPCTLSetPrintLevel	设置打印输出级别	0
JX_3tAPCTLSetARRRelaxTol	设置光子的磨光阈值	1.0e-6
JX_3tAPCTLSetARRRelaxType	设置光子的磨光类型	RELAX_AMG
JX_3tAPCTLSetThetaWCE	设置电子的弱耦合因子	1.0e-4
JX_3tAPCTLSetThresholdWCE	设置电子的弱耦合阈值	0.5
JX_3tAPCTLSetThetaDDR	设置光子的对角占优因子	0.9
JX_3tAPCTLSetThetaDDE	设置电子的对角占优因子	0.9
JX_3tAPCTLSetThetaDDI	设置离子的对角占优因子	0.9
JX_3tAPCTLSetThresholdDDR	设置光子的对角占优阈值	1.0
JX_3tAPCTLSetThresholdDDE	设置电子的对角占优阈值	1.0
JX_3tAPCTLSetThresholdDDI	设置离子的对角占优阈值	1.0
JX_3tAPCTLSetUseFixedModeR	设置光子部分采用固定的求解模式	1

§3.7 退化为单层情形

包含 Jacobi、BDS、Gauss-Seidel、SOR、SSOR、ILU、GE 等所有的磨光算法都可以退化为单层算法使用, 如设置 AMG 法的最大网格层数为 1, 此时 AMG 法即退化为指定的磨

参考文献

- [1] J. W. Ruge, K. Stüben, Algebraic Multigrid, in Multigrid Methods, Frontiers Appl. Math. 3, S. F. McCormick editor, SIAM, Philadelphia, 73-130, 1987.
- [2] K. Stüben, Algebraic multigrid (AMG): an introduction with applications, in Multigrid, U. Trottenberg, C. W. Oosterlee, A. Schuller, Academic Press, 413-532, 2001.
- [3] 徐小文, 莫则尧, 安恒斌, 求解二维三温辐射扩散方程组的一种代数两层网格迭代方法, 计算物理, 26 (1): 1-8, 2009.
- [4] 周志阳, 徐小文, 舒适, 冯春生, 莫则尧, 二维三温辐射扩散方程两层预条件子的自适应求解, 计算物理, 29 (4): 475-483, 2012.
- [5] X. Yue, S. Shu, X. Xu, Z. Zhou, An adaptive combined preconditioner with applications in radiation diffusion equations, Commun. Comput. Phys., 18 (5): 1313-1335, 2015.
- [6] X. Xu, Z. Mo, Algebraic interface based coarsening AMG preconditioner for multi-scale sparse matrices with applications to radiation hydrodynamics computation, Numer. Linear Algebra Appl., 24: e2078, 2017.
- [7] 徐小文, 莫则尧, 安恒斌, 求解大规模稀疏线性代数方程组序列的自适应 AMG 预条件策略, 中国科学: 信息科学, 46: 1411-1420, 2016.
- [8] X. Xu, Z. Mo, X. Yue, H. An, S. Shu, α Setup-AMG: an adaptive setup based parallel AMG solver for sequence of sparse linear systems, CCF Trans. High Perform. Comput., 2: 98-110, 2020.
- [9] X. Yue, S. Zhang, X. Xu, S. Shu, W. Shi, Algebraic multigrid block preconditioning for multi-group radiation diffusion equations, Commun. Comput. Phys., 29 (3): 831-852, 2021.
- [10] V. E. Henson, U. M. Yang, BoomerAMG: a parallel algebraic multigrid solver and preconditioner, Appl. Numer. Math., 41: 155-177, 2002.
- [11] Y. Saad, Iterative methods for sparse linear systems, 2nd Edition, SIAM, Philadelphia, 2003.
- [12] Z. Mo, A. Zhang, X. Cao, Q. Liu, X. Xu, H. An, W. Pei, S. Zhu, JASMIN: a parallel software infrastructure for scientific computing, Front. Comput. Sci. China, 4 (4): 480-488, 2010.
- [13] Q. K. Liu, Z. Y. Mo, A. Q. Zhang, Z. Yang, JAUMIN: a programming framework for large-scale numerical simulation on unstructured meshes, CCF Trans. High Perform. Comput., 1: 35-48, 2019.

- [14] U. Yang, On the use of relaxation parameters in hybrid smoothers, *Numer. Linear Algebra Appl.*, 11: 155-172, 2004.
- [15] A. Baker, R. Falgout, T. Kolev, U. Yang, Multigrid smoothers for ultra-parallel computing, *SIAM J. Sci. Comput.*, 33: 2864-2887, 2011.
- [16] A. J. Cleary, R. D. Falgout, V. E. Henson, J. E. Jones, Coarse-grid selection for parallel algebraic multigrid, *Lect. Notes Comput. Sci.*, 1457: 104-115, 1998.
- [17] M. Luby, A simple parallel algorithm for the maximal independent set problem, *SIAM J. Comput.*, 15: 1036-1053, 1986.
- [18] M. T. Jones, P. E. Plassman, A parallel graph coloring heuristic, *SIAM J. Sci. Comput.*, 14: 654-669, 1993.
- [19] H. de Sterck, U. M. Yang, J. J. Heys, Reducing complexity in parallel algebraic multigrid preconditioners, *SIAM J. Matrix Anal. Appl.*, 27: 1019-1039, 2006.
- [20] 徐小文, 莫则尧, 一种新的并行代数多重网格粗化算法, *计算数学*, 27 (3): 325-336, 2005.
- [21] Z. Mo, X. Xu, Relaxed RS0 or CLJP coarsening strategy for parallel AMG methods, *Parallel Computing*, 33 (3): 174-185, 2007.
- [22] R. D. Falgout, J. B. Schroder, Non-Galerkin coarse grids for algebraic multigrid, *SIAM J. Sci. Comput.*, 36: C309-C334, 2014.
- [23] J. Xu, L. Zikatanov, Algebraic multigrid methods, *Acta Numer.*, 26: 591-721, 2017.
- [24] 徐小文, 并行代数多重网格算法: 大规模计算应用现状与挑战, *数值计算与计算机应用*, 40 (4): 243-260, 2019.